



成员 B 汇报讲稿

第 14 页：Mode B 模型训练工作台

下面由我介绍 Mode B，真实训练工作台。

页面训练部分主要功能有数据集选择、网络搭建、参数配置和训练控制。用户可以使用内置数据，也可以上传 CSV 或图片。训练前还会检查输入维度、网络层和输出类别数。

第 15 页：训练任务执行流程

用户点击开始训练后，前端会把数据集、网络结构和超参数提交给 Spring。

因为模型训练需要在python上完成所以还需要python worker，可以看下这个模型训练启动的全流程。

`job.writeRequest()` 先把配置写入 `request.json`，供 Python 读取。`writeControl` 把任务设为运行中。`ProcessBuilder` 传入 Python 路径、worker 脚本和请求文件，启动独立进程。`redirectErrorStream(true)` 把错误日志合并到输出。最后两个后台任务分别读取事件和等待进程退出，避免 Spring 被训练阻塞。

第 16 页：训练过程实时展示

训练时，页面会实时显示 epoch、loss、准确率、学习率和梯度变化。

看下这里的代码，

`snapshot_trainable_parameters` 先保存更新前的参数。`loss.backward()` 计算梯度，`collect_layer_backprop_stats` 按层统计梯度。执行 `optimizer.step()` 后，再根据参数差值计算 `update_norm`。最后把轮次、损失、学习率和各层数据组成事件，由 Spring 推送到页面。

第 17 页：网络层梯度分析

点击网络层图标，可以查看这一层的梯度曲线、更新曲线和直方图。`grad_norm` 表示总体梯度大小，`update_norm` 表示参数实际改变的幅度。

每收到一次 backprop 事件, `current.push` 就保存 step、梯度、更新量、权重和直方图。数据按 `layerId` 分开, 避免不同层混在一起。

`slice(-80)` 只保留最近 80 个点, 控制历史数据的内存占用。

第 18 页: 数据集与训练结果存储

CSV 上传时需要指定标签列和类别数, 图片 ZIP 则按目录识别类别。内置数据集公开使用, 上传数据集按照用户隔离。

收到 `test_result` 且尚未保存时, 代码才调用 `saveCheckpoint`, 防止重复记录。`.pt` 保存模型权重, `TrainingCheckpoint` 则把文件路径、网络层、超参数、数据划分、指标和任务状态存入数据库。查询时先读取数据库, 需要测试或推理时再加载权重。

第 19 页: Checkpoint 测试集评估

训练完成后, 用户可以选择历史 Checkpoint, 单独重新运行测试集。

`torch.load` 读取权重文件, 再根据原来的 layers 和类别数创建相同模型。`load_state_dict` 恢复参数, `evaluate` 只做前向计算, 不更新模型。最后通过 `test_result` 返回 loss、accuracy 和预测样本。

第 20 页: 历史训练实验对比

实验对比页面按照数据集整理当前用户的历史训练记录, 可以比较网络结构、超参数、数据划分和各项指标。

页面通过 GET 接口按 `datasetId` 查询 Checkpoint。返回的 `TrainingCheckpointSummary` 包含网络层、训练配置、指标历史、状态和已完成轮数。前端据此绘制曲线、还原网络, 并标记未完成或异常任务。

第 21 页: 单样本逐层推理

用户先选择已完成的 Checkpoint, 再从对应数据集中选择一条样本。图片直接预览, CSV 展示原始字段。后端加载 `.pt` 权重, 返回预测结果和每层 activation。

`load_state_dict` 恢复权重, `model.eval()` 切换到推理模式, 避免 Dropout 等层继续随机工作。 `torch.no_grad()` 关闭梯度记录。 `infer_with_activations` 在前向传播时收集各层输出, 再把预测和 `activations` 返回前端逐层播放。

第 22 页：多人训练协作

聊天室使用 WebSocket 实现实时通信。Angular 前端通过浏览器原生的 `WebSocket` API 连接 Spring Boot, 后端则使用 `TextWebSocketHandler` 管理连接和广播消息。

这里实际分成两条 WebSocket 连接: `/api/training/collaboration` 负责聊天消息、历史记录和在线成员; `/api/training/collaboration/stream` 是只读的训练数据通道, 负责推送进度、指标和日志。两类数据分开传输, 聊天消息不会影响训练状态更新。前端收到消息后, 通过 `BehaviorSubject` 更新聊天列表和在线成员, 页面不需要手动刷新。

第一段代码区分“加入”和“创建”。加入时如果房间不存在, 就关闭连接, 不会自动创建。用户离开后, 后端从 `sessions` 移除连接; 集合为空时, 再从 `rooms` 删除房间。

以上就是我负责的 Mode B 部分。